Midterm Exam

- Due Mar 11 at 10pm
- Points 100
- Questions 10
- Available Mar 10 at 12am - Mar 11 at 10pm
- Time Limit 75 Minutes

Instructions

This exam is auto-graded as much as possible. All problems that are auto-marked incorrect will be manually examined by the teaching staff, and all auto-graded problems have space to show work. If an auto-graded problem is marked wrong and you provide correct work--that is, your error is demonstrably one of data entry--your points will be returned. You may forgo showing work; however, no points will be returned on problems where work is not provided.

You are permitted a simple calculator to aid you with arithmetic.

In all problems related to ARM assembly, integer values are always to be assumed to be 8 bytes each. This will not be restated in the problems.

This quiz was locked Mar 11 at 10pm.

Attempt History

	Attempt	Time	Score
LATEST	Attempt 1	75 minutes	74.51 out of 100

ⓘ Correct answers are hidden.

Score for this quiz: 74.51 out of 100
Submitted Mar 11 at 3:50pm
This attempt took 75 minutes.

LEGv8 Reference Data Card ("Green Card") 1. Pull along perforation to separate card 2. Fold bottom side (columns 3 and 4) together

①

LEGv8 Reference Data

CORE INSTRUCTION SET in Alphabetical Order by Mnemonic

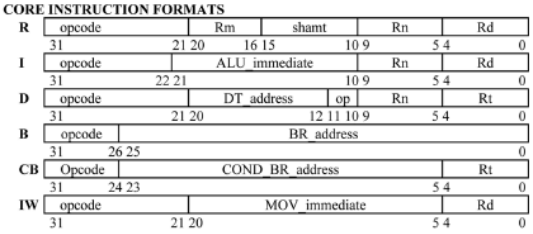
NAME, MNEMONIC		FOR-MAT	OPCODE (9) (Hex)	OPERATION (in Verilog)	Notes
ADD	ADD	R	458	R[Rd] = R[Rn] + R[Rm]	
ADD Immediate	ADDI	I	488-489	R[Rd] = R[Rn] + ALUImm	(2,9)
ADD Immediate & Set flags	ADDIS	I	588-589	R[Rd] , FLAGS = R[Rn] + ALUImm	(1,2,9)
ADD & Set flags	ADDS	R	558	R[Rd] , FLAGS = R[Rn] + R[Rm]	(1)
AND	AND	R	450	R[Rd] = R[Rn] & R[Rm]	
AND Immediate	ANDI	I	490-491	R[Rd] = R[Rn] & ALUImm	(2,9)
AND Immediate & Set flags	ANDIS	I	790-791	R[Rd] , FLAGS = R[Rn] & ALUImm	(1,2,9)
AND & Set flags	ANDS	R	750	R[Rd] , FLAGS = R[Rn] & R[Rm]	(1)
Branch	B	B	0A0-0BF	PC = PC + BranchAddr	(3,9)
Branch conditionally	B.cond	CB	2A0-2A7	if(FLAGS==cond) PC = PC + CondBranchAddr	(4,9)
Branch with Link	BL	B	4A0-4BF	R[30] = PC + 4; PC = PC + BranchAddr	(3,9)
Branch to Register	BR	R	6B0	PC = R[Rt]	
Compare & Branch if Not Zero	CBNZ	CB	5A8-5AF	if(R[Rt]!=0) PC = PC + CondBranchAddr	(4,9)
Compare & Branch if Zero	CBZ	CB	5A0-5A7	if(R[Rt]==0) PC = PC + CondBranchAddr	(4,9)
Exclusive OR	EOR	R	650	R[Rd] = R[Rn] ^ R[Rm]	
Exclusive OR Immediate	EORI	I	690-691	R[Rd] = R[Rn] ^ ALUImm	(2,9)
LoaD Register Unscaled offset	LDUR	D	7C2	R[Rt] = M[R[Rn] + DTAddr]	(5)
LoaD Byte Unscaled offset	LDURB	D	1C2	R[Rt]={56'b0, M[R[Rn] + DTAddr](7:0)}	(5)
LoaD Half Unscaled offset	LDURH	D	3C2	R[Rt]={48'b0, M[R[Rn] + DTAddr] (15:0)}	(5)
LoaD Signed Word Unscaled offset	LDURSW	D	5C4	R[Rt] ={ 32{ M[R[Rn] + DTAddr][31]}, M[R[Rn] + DTAddr] (31:0)}	(5)
LoaD eXclusive Register	LDXR	D	642	R[Rd] = M[R[Rn] + DTAddr]	(5,7)
Logical Shift Left	LSL	R	69B	R[Rd] = R[Rn] << shamt	
Logical Shift Right	LSR	R	69A	R[Rd] = R[Rn] >>> shamt	
MOVe wide with Keep	MOVK	IM	794-797	R[Rd] (Instruction[22:21]*16: Instruction[22:21]*16-15) = MOVImm	(6,9)
MOVe wide with Zero	MOVZ	IM	694-697	R[Rd] = { MOVImm << (Instruction[22:21]*16) }	(6,9)
Inclusive OR	ORR	R	550	R[Rd] = R[Rn] | R[Rm]	
Inclusive OR Immediate	ORRI	I	590-591	R[Rd] = R[Rn] | ALUImm	(2,9)
STore Register Unscaled offset	STUR	D	7C0	M[R[Rn] + DTAddr] = R[Rt]	(5)
STore Byte Unscaled offset	STURB	D	1C0	M[R[Rn] + DTAddr](7:0) = R[Rt](7:0)	(5)
STore Half Unscaled offset	STURH	D	3C0	M[R[Rn] + DTAddr](15:0) = R[Rt](15:0)	(5)
STore Word Unscaled offset	STURW	D	5C0	M[R[Rn] + DTAddr](31:0) = R[Rt](31:0)	(5)
STore eXclusive Register	STXR	D	640	M[R[Rn] + DTAddr] = R[Rt]; R[Rm] = (*atomic*) ? 0 : 1	(5,7)
SUBtract	SUB	R	658	R[Rd] = R[Rn] – R[Rm]	
SUBtract Immediate	SUBI	I	688-689	R[Rd] = R[Rn] – ALUImm	(2,9)
SUBtract Immediate & Set flags	SUBIS	I	788-789	R[Rd] , FLAGS = R[Rn] – ALUImm	(1,2,9)
SUBtract & Set flags	SUBS	R	758	R[Rd] , FLAGS = R[Rn] – R[Rm]	(1)

(1) FLAGS are 4 condition codes set by the ALU operation: Negative, Zero, oVerflow, Carry
(2) ALUImm = { 52'b0, ALU_immediate }
(3) BranchAddr = { 36{BR_address [25]}, BR_address, 2'b0 }
(4) CondBranchAddr = { 43{COND_BR_address [25]}, COND_BR_address, 2'b0 }
(5) DTAddr = { 55{DT_address [8]}, DT_address }
(6) MOVImm = { 48'b0, MOV_immediate }
(7) Atomic test&set pair; R[Rm] = 0 if pair atomic, 1 if not atomic
(8) Operands considered unsigned numbers (vs. 2's complement)
(9) Since I, B, and CB instruction formats have opcodes narrower than 11 bits, they occupy a range of 11-bit opcodes
(10) If neither is operand a NaN and Value1 == Value2, FLAGS = 4'b0110;
If neither is operand a NaN and Value1 < Value2, FLAGS = 4'b1000;
If neither is operand a NaN and Value1 > Value2, FLAGS = 4'b0010;
If an operand is a Nan, operands are unordered

ARITHMETIC CORE INSTRUCTION SET ②

NAME, MNEMONIC		FOR-MAT	OPCODE/ SHAMT (Hex)	OPERATION (in Verilog)	Notes
Floating-point ADD Single	FADDS	R	0F1 / 0A	S[Rd] = S[Rn] + S[Rm]	
Floating-point ADD Double	FADDD	R	0F3 / 0A	D[Rd] = D[Rn] + D[Rm]	
Floating-point CoMPare Single	FCMPS	R	0F1 / 08	FLAGS = (S[Rn] vs S[Rm])	(1,10)
Floating-point CoMPare Double	FCMPD	R	0F3 / 08	FLAGS = (D[Rn] vs D[Rm])	(1,10)
Floating-point DIVide Single	FDIVS	R	0F1 / 06	S[Rd] = S[Rn] / S[Rm]	
Floating-point DIVide Double	FDIVD	R	0F3 / 06	D[Rd] = D[Rn] / D[Rm]	
Floating-point MULtiply Single	FMULS	R	0F1 / 02	S[Rd] = S[Rn] * S[Rm]	
Floating-point MULtiply Double	FMULD	R	0F3 / 02	D[Rd] = D[Rn] * D[Rm]	
Floating-point SUBtract Single	FSUBS	R	0F1 / 0E	S[Rd] = S[Rn] – S[Rm]	
Floating-point SUBtract Double	FSUBD	R	0F3 / 0E	D[Rd] = D[Rn] – D[Rm]	
LoaD Single floating-point	LDURS	R	7C2	S[Rt] = M[R[Rn] + DTAddr]	(5)
LoaD Double floating-point	LDURD	R	7C0	D[Rt] = M[R[Rn] + DTAddr]	(5)
MULtiply	MUL	R	4D8 / 1F	R[Rd] = (R[Rn] * R[Rm]) (63:0)	
Signed DIVide	SDIV	R	4D6 / 02	R[Rd] = R[Rn] / R[Rm]	
Signed MULtiply High	SMULH	R	4DA	R[Rd] = (R[Rn] * R[Rm]) (127:64)	
STore Single floating-point	STURS	R	7E2	M[R[Rn] + DTAddr] = S[Rt]	(5)
STore Double floating-point	STURD	R	7E0	M[R[Rn] + DTAddr] = D[Rt]	(5)
Unsigned DIVide	UDIV	R	4D6 / 03	R[Rd] = R[Rn] / R[Rm]	(8)
Unsigned MULtiply High	UMULH	R	4DE	R[Rd] = (R[Rn] * R[Rm]) (127:64)	(8)

CORE INSTRUCTION FORMATS



PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
CoMPare	CMP	FLAGS = R[Rn] – R[Rm]
CoMPare Immediate	CMPI	FLAGS = R[Rn] – ALUImm
LoaD Address	LDA	R[Rd] = R[Rn] + DTAddr
MOVe	MOV	R[Rd] = R[Rn]

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
X0 – X7	0-7	Arguments / Results	No
X8	8	Indirect result location register	No
X9 – X15	9-15	Temporaries	No
X16 (IP0)	16	May be used by linker as a scratch register; other times used as temporary register	No
X17 (IP1)	17	May be used by linker as a scratch register; other times used as temporary register	No
X18	18	Platform register for platform independent code; otherwise a temporary register	No
X19-X27	19-27	Saved	Yes
X28 (SP)	28	Stack Pointer	Yes
X29 (FP)	29	Frame Pointer	Yes
X30 (LR)	30	Return Address	Yes
XZR	31	The Constant Value 0	N.A.

© 2017 Elsevier Inc. All rights reserved. From Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface ARM® Edition*

OPCODES IN NUMERICAL ORDER BY OPCODE ③

Instruction Mnemonic	Format	Opcode Width (bits)	Opcode Binary	Shamt Binary	11-bit Opcode Range (1) Start (Hex)	End (Hex)
B	B	6	000101		0A0	0BF
FMULS	R	11	00011110001	000010	0F1	
FDIVS	R	11	00011110001	000110	0F1	
FCMPS	R	11	00011110001	001000	0F1	
FADDS	R	11	00011110001	001010	0F1	
FSUBS	R	11	00011110001	001110	0F1	
FMULD	R	11	00011110011	000010	0F3	
FDIVD	R	11	00011110011	000110	0F3	
FCMPD	R	11	00011110011	001000	0F3	
FADDD	R	11	00011110011	001010	0F3	
FSUBD	R	11	00011110011	001110	0F3	
STURB	D	11	00111000000		1C0	
LDURB	D	11	00111000010		1C2	
B.cond	CB	8	01010100		2A0	2A7
STURH	D	11	01111000000		3C0	
LDURH	D	11	01111000010		3C2	
AND	R	11	10001010000		450	
ADD	R	11	10001011000		458	
ADDI	I	10	1001000100		488	489
ANDI	I	10	1001001000		490	491
BL	B	6	100101		4A0	4BF
SDIV	R	11	10011010110	000010	4D6	
UDIV	R	11	10011010110	000011	4D6	
MUL	R	11	10011011000	011111	4D8	
SMULH	R	11	10011011010		4DA	
UMULH	R	11	10011011110		4DE	
ORR	R	11	10101010000		550	
ADDS	R	11	10101011000		558	
ADDIS	I	10	1011000100		588	589
ORRI	I	10	1011001000		590	591
CBZ	CB	8	10110100		5A0	5A7
CBNZ	CB	8	10110101		5A8	5AF
STURW	D	11	10111000000		5C0	
LDURSW	D	11	10111000100		5C4	
STURS	R	11	10111100000		5E0	
LDURS	R	11	10111100010		5E2	
STXR	D	11	11001000000		640	
LDXR	D	11	11001000010		642	
EOR	R	11	11001010000		650	
SUB	R	11	11001011000		658	
SUBI	I	10	1101000100		688	689
EORI	I	10	1101001000		690	691
MOVZ	IM	9	110100101		694	697
LSR	R	11	11010011010		69A	
LSL	R	11	11010011011		69B	
BR	R	11	11010110000		6B0	
ANDS	R	11	11101010000		750	
SUBS	R	11	11101011000		758	
SUBIS	I	10	1111000100		788	789
ANDIS	I	10	1111001000		790	791
MOVK	IM	9	111100101		794	797
STUR	D	11	11111000000		7C0	
LDUR	D	11	11111000010		7C2	
STURD	R	11	11111100000		7E0	
LDURD	R	11	11111100010		7E2	

(1) Since I, B, and CB instruction formats have opcodes narrower than 11 bits, they occupy a range of 11-bit opcodes, e.g., the 6-bit B format occupies 32 (2^5) 11-bit opcodes.

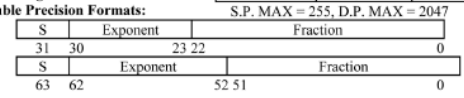
IEEE 754 FLOATING-POINT STANDARD ④

$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$
where Single Precision Bias = 127,
Double Precision Bias = 1023

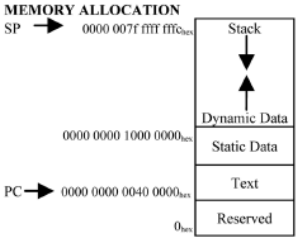
IEEE 754 Symbols

Exponent	Fraction	Object
0	0	± 0
0	≠ 0	± Denorm
1 to MAX - 1	anything	± F1. Pt. Num.
MAX	0	± ∞
MAX	≠ 0	NaN

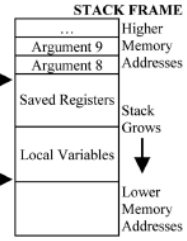
IEEE Single Precision and Double Precision Formats: S.P. MAX = 255, D.P. MAX = 2047



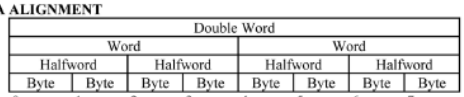
MEMORY ALLOCATION



STACK FRAME

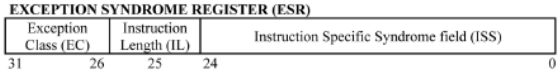


DATA ALIGNMENT



Value of three least significant bits of byte address (Big Endian)

EXCEPTION SYNDROME REGISTER (ESR)



EXCEPTION CLASS

EC	Class	Cause of Exception	Number	Name	Cause of Exception
0	Unknown	Unknown	34	PC	Misaligned PC exception
7	SIMD	SIMD/FP registers disabled	36	Data	Data Abort
14	FPE	Illegal Execution State	40	FPE	Floating-point exception
17	Sys	Supervisor Call Exception	52	WPT	Data Breakpoint exception
32	Instr	Instruction Abort	56	BKPT	SW Breakpoint Exception

SIZE PREFIXES AND SYMBOLS

SIZE	PREFIX	SYMBOL	SIZE	PREFIX	SYMBOL
10^3	Kilo-	K	2^{10}	Kibi-	Ki
10^6	Mega-	M	2^{20}	Mebi-	Mi
10^9	Giga-	G	2^{30}	Gibi-	Gi
10^{12}	Tera-	T	2^{40}	Tebi-	Ti
10^{15}	Peta-	P	2^{50}	Pebi-	Pi
10^{18}	Exa-	E	2^{60}	Exbi-	Ei
10^{21}	Zetta-	Z	2^{70}	Zebi-	Zi
10^{24}	Yotta-	Y	2^{80}	Yobi-	Yi
10^{-3}	milli-	m	10^{-15}	femto-	f
10^{-6}	micro-	μ	10^{-18}	atto-	a
10^{-9}	nano-	n	10^{-21}	zepto-	z
10^{-12}	pico-	p	10^{-24}	yocto-	y

© 2017 Elsevier Inc. All rights reserved. From Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface ARM® Edition*

LEGv8 Reference Data Card ("Green Card") 1. Pull along perforation to separate card 2. Fold bottom side (columns 3 and 4) together

Question 1
18 / 25 pts

Write an ARMv8 (please restrict yourself to the LEGv8 subset of ARMv8) procedure which takes a single parameter, the base address of an array, and returns the value of the smallest element in that array. The array conforms to the following conditions (ensuring this is the responsibility of the caller, so you do not need to check!):

- Each element of the array is a 64-bit integer;
- The array contains only positive numbers (the elements would be **unsigned** if it were a C array);
- Each of the values is unique (no repeats);
- All of the values are non-zero;
- A zero markes the end of the array, but (see previous bullet) is not considered an element *in* the array; and
- the array is not empty (there will always be at least one value *before* the zero).

Again, you may assume these conditions are true. Do not check for them!

We will not run your code. We do not expect perfection. If you have a working idea that shows that you understand how to write code in ARM assembly, you'll get most, if not all, of the credit for this problem.

Hints:

- Write a C function first and "compile" it to assembly.
- The instructor's solution has 9 instructions. A shorter solution may be possible, but if your solution is significantly longer than that, you're probably doing something wrong.
- Rather than keeping an index into the array and converting it to an offset each iteration, you can save several instructions by simply incrementing the base address by 8. Then you never have to worry about indices *or* offsets!

Your Answer:

// X0 = base // X2 = smallest // X3 = i // X4 = current element

SMALLEST:

LDUR X2, [X0, #0]

ADDI X3, XZR, #1

LOOP:

LSL X10, X3, #3

ADD X10, X10, X0

LDUR X4, [X10, #0]

CMN X4, #1

B.Eq Exit

ADD X2, X4, XRZ

ADDI X3, X3, #1
B LOOP

Exit:

ADD X0, X2, XZR

BR LR // return

main issue is that the comparison and sentinel check are not implemented correctly, but the overall direction is reasonable.

Question 2
10 / 10 pts

You are designing a floating point coprocessor which will execute floating point instructions 5 times as fast as the regular processor with zero overhead (that is, it finishes five floating point instructions in the time the main processor completes one). Given a workload where the main processor spends 25% of the execution time on floating-point instructions, what is the speedup achieved by adding a coprocessor that executes floating-point instructions, with the main processor handling other instructions? Assume that they cannot execute instructions in parallel.

Please give the speedup with **two digits** of precision after the decimal point.

1.25

Question 3
0 / 0 pts

For a chance at partial credit, please show your work for the previous problem.

Your Answer:

T improved = (T affected / Improved factor) + T unaffected

Here to find the T improved we have -
T affected = 25% (Since main processor spends 25% of the execution time on floating point instructions)

T unaffected = 75% (100 - 25 = 75)
Improved factor = 5 (Since the coprocessor is 5 times as fast as the regular processor)

Thus, T improved = (0.25/5) + 0.75 = 0.8 = 80%

Thus the new time it takes becomes 80% of the original time. The speedup is 100/80 = 1.25 times.

Question 4
10 / 10 pts

Given the same workload as the previous problem, what is the smallest amout of speedup that is impossible to achieve, no matter how fast your coprocessor becomes?

Please give your answer with **two digits** of precision after the decimal point.

1.33

Question 5
0 / 0 pts

For a chance at partial credit, please show your work for the previous problem.

Your Answer:

The coprocessor only affects floating point instructions. Thus even if the coprocessor did the floating point instructions instantly, the main processor will still have to do the other 75% of the instructions.

So lets say the coprocessor does the floating point instructions instantly,
T improved = (0.25/infinity) + 0.75; (Here 0.25/infinity becomes 0)
Thus, T improved = 75% the original time.

Speedup = 100/75 = 1.33 times. This is the max speedup.

Partial

Question 6
6.67 / 10 pts

Fill in the blanks in the table below. The first row is complete, giving you a full example. All hexadecimal numbers should give two bytes.

Value in decimal	Same value in 16-bit two's complement hexadecimal (signed number)
9385	0x24A9

1111	0x 457
3346	0x0D12
-1724	0x 6BC

Answer 1:
457
Answer 2:
3346
Answer 3:
6BC
Partial

Question 7
16.67 / 25 pts

Fill in the blanks by encoding the instruction in front of the blank in this code snippet into ARMv8 machine code. Use binary (base 2) to write your encoded instructions in the blanks.

Carefully count bits in your input! If you omit a bit in an otherwise correct answer, you will not receive full credit.

Canvas will auto-grade your entries. The auto-grader will give credit only for a perfect answer, where "perfect" means an exact string match with the answer entered when creating the exam. We will manually re-grade answers marked incorrect for partial credit. **Answers that the auto-grader will consider correct consist of a string of exactly 32 zeros and ones with no other characters or spaces.**

```
selection_sort:
  SUBI SP, SP, #24
  STUR LR, [SP, #0]
  STUR X0, [SP, #8]

  STUR X1, [SP, #16]        1111100000000001


  ADD X19, XZR, XZR

  SUBI X20, X1, #1          1101000100000000


selection_loop:
  SUBS X9, X19, X20
```

```
  B.EQ end_selection_loop
  LSL X9, X19, #3
  ADD X0, X0, X9
  ADD X21, X0, XZR
  SUB X1, X1, X19
  BL find_smallest
  LSL X0, X0, #3
  ADD X1, X21, X0
  ADD X0, X21, XZR
  BL swap
  LDUR X1, [SP, #16]
  LDUR X0, [SP, #8]
  ADDI X19, X19, #1

  B selection_loop

end_selection_loop:
  LDUR LR, [SP, #0]
  ADDI SP, SP, #24
  BR LR
done:
```

0001010000000000(

Answer 1:
11111000000000010000001110000001
Answer 2:
11010001000000000000010000110100
Answer 3:
00010100000000000000000111

Question 8
3.5 / 0 pts

Give the fields (in binary or hex) for each instruction that you encoded in the previous problem for consideration for partial credit. Please clearly denote which instruction and field you are presenting.

Your Answer:

// opcode -> type -> hex -> binary

STUR -> D type -> 7C0 -> 11111000000 [D type bits -> 11 opcode , 9 address, 2 op2, 5 Rn, 5 Rt] -> 000010000 (16) -> 00 -> 11100 (Sp is x28) -> 00001

SUBI -> I type -> 688-689 -> 1101000100 [I type bits -> 10 opcode, 12 immediate, 5 Rn, 5 Rd] -> 000000000001 (1)-> 00001 (x1) -> 10100

B -> B type -> 0A0-0BF -> 000101 [B type bits -> 6 opcode, 26 address] -> Branch Address -> 7 -> 00000000000000000111

Blank 3 (+3.5)

Incorrect

Question 9
0 / 20 pts

Decode the following instructions. If the instruction is a branch--since the branch target name is lost--simply write the number of instructions (positive or negative) that the jump moves control in the position where you would ordinarily write the branch target name; you do not need to put a hash before the number of instructions in this case. All integers (and register numbers) in your output should be written in base 10.

Canvas will auto-grade your entries. The auto-grader will give credit only for a perfect answer, where "perfect" means an exact string match with the answer entered by the test writer. We will manually re-grade answers marked incorrect for partial or full credit. Answers that the auto-grader will consider correct have all letters upper case, use exactly one space after the mnemonic and each comma, use a hash before all literals, and all literals will be in base 10.

The example answer is formatted per the auto-grader's expectations.

`10010001000010000000001111101010` `ADDI X10, XZR, #512`

`11001011000111110000000101001100` SUB X12, X10, X31

`10110100000000000000000010101100` CBZ X12,

`11010011011000000000110100101100`

Answer 1:
SUB X12, X10, X31
Answer 2:
CBZ X12,
Answer 3:
(You left this blank)

Question 10
9.67 / 0 pts

Show your work for the previous problem for consideration for partial credit.

Your Answer:

1) First 11 digits -> convert to hexadecimal which becomes 658 which is SUB. Which is R type (11 opcode, 5 Rm, 6 shamt, 5 Rn, 5 Rd)
Rd is last 5 digits -> X12,
Rn is last 10 to 5 digits -> X10
Rm is 5 digits after opcode -> X31

2) First 11 digits -> 5A0 which is CBZ which is CB Type (8 op, 19 address, 5 Rt)
Rt is last 5 digits -> X12
Label -> 5 in binary ->

3)

Instruction 1: Student fully correct (+6.67). Instruction 2: correct opcode and rt, missing immediate (+3).

Quiz Score: 74.51 out of 100